Name – Sai Nikitha NSR
Day – 7 Assignment
Date – 4th May, 2026
Mail – sainikinsr@gmail.com

OOPS IN PROJECT

Bus Booking System — Full Stack Application

Angular .NET Web API · PostgreSQL

Assignment Task

Create a document that explains your project based on what we learned in OOPS today.
If the concepts are already done — display them and explain what you understand better now.
If not done already — add those points and explain how it makes it better.

1. Project Overview

Bus Booking System is an end-to-end web-based application that mimics actual bus travels booking processes in the real world. This application has been developed using **Angular (frontend), .NET 10 Web API (backend), and PostgreSQL (database).**

There are three types of users supported by the application – **Customers, Operators, and Admin**.

1.1 Folder Structure

The backend is cleanly organised into dedicated folders, each with a single responsibility:

Folder	Purpose
Backend/Models/	C# entity classes that map to PostgreSQL tables (Booking, Bus, User, Feedback, …)
Backend/Controllers/	ASP.NET controllers that expose REST API endpoints
Backend/Services/	Business-logic classes (EmailService) — separated from controllers
Backend/Interfaces/	C# interfaces that define service contracts (IEmailService) ← NEW
Backend/Data/	Entity Framework AppDbContext — database session / DbSet definitions
Frontend/src/app/pages/	Angular page components (Home, Login, Seat, Payment, Ticket, …)
Frontend/src/app/components/	Shared UI components (Navbar)
Frontend/src/app/services/	Angular services that call the backend API

Name – Sai Nikitha NSR
Day – 7 Assignment

Date – 4th May, 2026
Mail – sainikinsr@gmail.com

2. OOP Concepts Learned Today

The below listed fundamental concepts of OOP were discussed in today's lecture:

- **Encapsulation** – encapsulating behavior and state in an object class by accessing variables through properties
- **Abstraction** – simplifying code by hiding unnecessary details from the users
- **Inheritance** – creating classes by inheriting behaviors of existing classes
- **Polymorphism** – overloading methods (same name but differing parameters) and overriding methods (overwriting inherited behaviors)
- **Interface** – defining a protocol for implementing service classes (decoupled systems, testable)
- **Coding standards** – using switch statements, concise methods, well organized folder structure

3. Encapsulation

Status: Already present in the project — further reinforced with understanding

3.1 What is Encapsulation?

Encapsulation means bundling data (fields) and behaviour (methods) that belong together inside a single class, and controlling how the outside world reads or writes that data using access modifiers and C# properties. An example taken from today's class, taking a capsule without knowing what medicine was given.

3.2 Where is it used? — Backend/Models/Booking.cs

Every model class in the Models/ folder applies encapsulation. Booking.cs is the clearest example — it groups every piece of data that belongs to a seat booking into one self-contained class.

```
// File: Backend/Models/Booking.cs
[Table("bookings")]
public class Booking
{
    [Column("id")]          public int     Id            { get; set; }
    [Column("user_id")]     public int     UserId        { get; set; }
    [Column("bus_id")]      public int     BusId         { get; set; }
    [Column("seat_number")] public int     SeatNumber    { get; set; }
    [Column("status")]      public string  Status        { get; set; } =
"Booked";
    [Column("lock_expiry")] public DateTime? LockExpiry  { get; set; }
    [Column("payment_status")] public string PaymentStatus { get; set; } =
"Pending";
    [Column("passenger_name")] public string? PassengerName { get; set; }
    [Column("age")]         public int?    Age           { get; set; }
    [Column("gender")]      public string? Gender        { get; set; }
    [Column("booking_group_id")] public int? BookingGroupId { get; set; }
}
```

3.3 How it helps

Where: Backend/Models/ — Booking.cs, Feedback.cs, Bus.cs, User.cs, BookingGroup.cs

What: Each class groups related properties using { get; set; } — no public fields, no scattered variables

Why: Data integrity is protected; you cannot accidentally write to an internal field from outside the class

Benefit: Objects are self-contained; a Booking object carries everything needed to describe one seat reservation

3.4 What I understand better now

Encapsulation, in my opinion, prior to today, meant 'making things private'. However, encapsulation means designing such that a class owns all the data related to it – the class Booking owns the representation of a seating reservation; nothing else can define its structure.

4. Abstraction

Status: Already present — controllers abstract away DB and SMTP details from the Angular frontend

4.1 What is Abstraction?

Abstraction means exposing only what is necessary and hiding internal implementation details. A caller does not need to know HOW something works — only WHAT it can do. For example, in today's class it was said how phones place a call when we press on call option but we don't have to know it's working.

4.2 Where is it used? — Backend/Controllers/ and Backend/Interfaces/IEmailService.cs

4.2a Controllers hide database logic

The Angular frontend calls a clean REST endpoint. It has no idea that Entity Framework or PostgreSQL are involved.

```
// Frontend sees only: POST /api/booking/lock
// Backend hides everything:
[HttpPost("lock")]
public IActionResult LockSeat([FromBody] Booking booking)
{
    var exists = context.Bookings.Any(b =>
        b.BusId == booking.BusId && b.SeatNumber == booking.SeatNumber &&
        (b.Status == "Booked" || b.Status == "Paid" ||
        (b.Status == "Locked" && b.LockExpiry > DateTime.UtcNow)));

    if (exists) return BadRequest("Seat already taken or locked");
    // ... lock and save
}
```

4.2b IEmailService — interface abstraction (NEW)

After today's session, the IEmailService interface was introduced. Controllers now depend only on the interface — they never see the SMTP/MailKit code.

```
// File: Backend/Interfaces/IEmailService.cs
namespace Backend.Interfaces
{
    public interface IEmailService
    {
        void SendBookingEmail(string to, string subject, string htmlBody);
        void SendBookingEmail(string to, string subject);   // overload
    }
}
```

4.3 How it helps

Where: BookingController, FeedbackController inject IEmailService — not EmailService

What: The interface hides Gmail SMTP, MailKit, app-password config, and HTML building

Why: If we switch from Gmail to SendGrid tomorrow, we create a new class implementing IEmailService — zero controller changes

Benefit: Clean separation between 'what the system can do' and 'how it does it'

5. Inheritance

Status: Already present — all controllers inherit from ASP.NET's ControllerBase

5.1 What is Inheritance?

Inheritance lets a child class acquire the properties and methods of a parent class, enabling code reuse and framework integration without rewriting boilerplate.

5.2 Where is it used? — Every Controller file

```
// File: Backend/Controllers/BookingController.cs
public class BookingController : ControllerBase   // inherits HTTP features
{
    // Inherited from ControllerBase:
    //   Ok(), BadRequest(), NotFound(), Conflict(), Unauthorized(), ...
    //   ModelState, HttpContext, RouteData, Request, Response
}

// Same pattern in every controller:
public class AdminController    : ControllerBase { ... }
public class FeedbackController : ControllerBase { ... }
public class BusController      : ControllerBase { ... }
public class OperatorController : ControllerBase { ... }
public class UserController     : ControllerBase { ... }
```

5.3 How it helps

Where: Backend/Controllers/ — all 6 controller classes

What: Each controller inherits Ok(), BadRequest(), NotFound() and the full HTTP pipeline from ControllerBase

Why: We write zero HTTP-handling code — the framework's parent class provides it all

Benefit: Thousands of lines of battle-tested HTTP code reused for free; we focus only on business logic

5.4 What I understand better now

Inheritance is not just for our own custom parent classes. ASP.NET's ControllerBase is a powerful parent built by Microsoft. Every time I write ': ControllerBase' I am inheriting an entire HTTP framework — that is real-world inheritance at scale.

6. Polymorphism

Status: Method overriding was implicit via framework. Method overloading was ADDED as a new enhancement today.

6.1 What is Polymorphism?

Polymorphism means 'many forms'. The same method name behaves differently depending on context — either via overloading (different parameter signatures) or overriding (redefining a parent class's method in a child class).

6.2 Method Overriding — ASP.NET Lifecycle (Already Present)

ASP.NET's ControllerBase exposes virtual lifecycle hooks. Any controller can override these to add cross-cutting behaviour (logging, auth checks, etc.).

```
// Example of overriding an ASP.NET lifecycle method:
public override void OnActionExecuting(ActionExecutingContext context)
{
    // Custom logic runs before every action in this controller
    base.OnActionExecuting(context);
}

// The framework calls the same method name on every controller,
// but each controller can give it a different implementation — polymorphism.
```

6.3 Method Overloading — EmailService (NEW Enhancement)

Today, a second overload of SendBookingEmail was added to both IEmailService and EmailService. The same method name now handles two different calling situations.

```
// File: Backend/Interfaces/IEmailService.cs
public interface IEmailService
```

```
{
    // Overload 1 — caller provides full HTML body
    void SendBookingEmail(string to, string subject, string htmlBody);

    // Overload 2 — convenience; body is auto-generated from the subject
    void SendBookingEmail(string to, string subject);
}

// File: Backend/Services/EmailService.cs
public void SendBookingEmail(string toEmail, string subject, string htmlBody)
{
    // ... builds MimeMessage and sends via Gmail SMTP
}

public void SendBookingEmail(string toEmail, string subject)
{
    var defaultBody = $"<div>...{subject}...</div>";
    SendBookingEmail(toEmail, subject, defaultBody);  // delegates to overload
1
}
```

6.4 How it helps

Where: Backend/Interfaces/IEmailService.cs and Backend/Services/EmailService.cs

What: Two versions of SendBookingEmail — full control (3 params) and quick notification (2 params)

Why: Callers choose the right version for their context without needing different method names

Benefit: Clean, readable API — the email service is flexible without being complicated

6.5 What I understand better now

I now understand that overloading is not just convenience — it is polymorphism. The compiler picks the right method at compile time based on how many arguments you pass. Overriding happens at runtime — the correct child-class version is chosen dynamically. Both are polymorphism, but they work at different stages.

7. Interfaces

Status: NEW — IEmailService interface created today; controllers updated to depend on the interface, not the concrete class

7.1 What is an Interface?

An interface is a contract. It declares method signatures (what must exist) without providing any implementation (how it works). Any class that 'implements' the interface must provide concrete implementations for every method declared.

Name – Sai Nikitha NSR
Day – 7 Assignment

Date – 4th May, 2026
Mail – sainikinsr@gmail.com

7.2 Files involved

File	Status	Role
Backend/Interfaces/IEmailService.cs	NEW	Declares the contract — two SendBookingEmail signatures
Backend/Services/EmailService.cs	UPDATED	Implements IEmailService; contains real Gmail SMTP code
Backend/Program.cs	UPDATED	Registers: AddScoped<IEmailService, EmailService>()
Backend/Controllers/BookingController.cs	UPDATED	Injects IEmailService — not EmailService directly
Backend/Controllers/FeedbackController.cs	UPDATED	Injects IEmailService — not EmailService directly

7.3 The Interface — full code

```
// File: Backend/Interfaces/IEmailService.cs
namespace Backend.Interfaces
{
    public interface IEmailService
    {
        // Full overload — caller provides HTML body
        void SendBookingEmail(string to, string subject, string htmlBody);

        // Convenience overload — body auto-generated
        void SendBookingEmail(string to, string subject);
    }
}
```

7.4 EmailService implements the interface

```
// File: Backend/Services/EmailService.cs
using Backend.Interfaces;

public class EmailService : IEmailService   // ← implements the contract
{
    public void SendBookingEmail(string toEmail, string subject, string
htmlBody)
    {
        // Real Gmail SMTP code using MailKit
        var message = new MimeMessage();
        message.From.Add(MailboxAddress.Parse(_fromEmail));
        message.To.Add(MailboxAddress.Parse(toEmail));
        message.Subject = subject;
        message.Body    = new TextPart("html") { Text = htmlBody };
        using var smtp = new SmtpClient();
        smtp.Connect("smtp.gmail.com", 587, false);
        smtp.Authenticate(_fromEmail, _appPassword);
        smtp.Send(message);
        smtp.Disconnect(true);
    }
```

```
    public void SendBookingEmail(string toEmail, string subject)
    {
        var defaultBody = $"<div>...{subject}...</div>";
        SendBookingEmail(toEmail, subject, defaultBody);
    }
}
```

7.5 DI registration in Program.cs

```
// File: Backend/Program.cs
// BEFORE (tightly coupled to concrete class):
builder.Services.AddScoped<EmailService>();

// AFTER (loosely coupled via interface):
builder.Services.AddScoped<IEmailService, EmailService>();
//  ↑ any IEmailService injected will receive an EmailService instance
```

7.6 Controllers use the interface, not the class

```
// File: Backend/Controllers/BookingController.cs
// BEFORE:
public class BookingController(AppDbContext context, EmailService
emailService) : ControllerBase

// AFTER — depends on interface, not implementation:
public class BookingController(AppDbContext context, IEmailService
emailService) : ControllerBase
//                                                  ^^ interface only

// Same change applied to FeedbackController
```

7.7 How it helps

Where: Interfaces/ folder → IEmailService; Services/ folder → EmailService

What: The interface separates the contract from the implementation

Why: BookingController should not care whether email is sent via Gmail or SendGrid or a mock

Benefit 1: Loose coupling — swap implementation without changing any controller

Benefit 2: Easy testing — inject a mock IEmailService in unit tests, no real email sent

Benefit 3: Open/Closed Principle — add new email providers by adding new classes, not editing existing ones

7.8 What I understand better now

An interface is a promise. When BookingController asks for an IEmailService, it is saying: 'give me anything that knows how to send a booking email — I do not care what is inside.' This makes the controller completely independent of how email is delivered. If the company changes email providers, no controller code changes — only the injected class changes.

Name – Sai Nikitha NSR
Day – 7 Assignment
Date – 4th May, 2026
Mail – sainikinsr@gmail.com

8. Coding Standards

Status: Partially present — short methods and clean structure already existed. Switch statement added today.

8.1 Short, Single-Purpose Methods (Already Present)

Every controller action does exactly one thing. The method name tells you what it does, and the body delivers that one thing — no mixing of concerns.

```
// File: Backend/Controllers/BookingController.cs
// Each method = one responsibility:

LockSeat()          — locks one seat for 5 minutes
CreateBooking()     — confirms a locked seat as booked
FinalizeGroup()     — groups individual bookings into one payment unit
PayGroup()          — marks all seats paid and sends confirmation email
GetGroupTicket()    — fetches full ticket details for a group
CancelBooking()     — cancels a booking and initiates refund if paid
GetBookedSeats()    — returns seat map data (booked/locked/gender)
```

8.2 Switch Statements (NEW Enhancement)

The UserController.Register method previously used an if statement to handle role-based logic. It has been refactored to a switch statement, making it ready for future roles and easier to read.

```
// File: Backend/Controllers/UserController.cs

// BEFORE — brittle if statement:
if (dto.Role == "Operator")
{
    _context.OperatorProfiles.Add(new OperatorProfile { ... });
    _context.SaveChanges();
}

// AFTER — clean switch statement:
switch (user.Role)
{
    case "Operator":
        var profile = new OperatorProfile
        {
            UserId       = user.Id,
            BusinessName = user.Name,
            IsApproved   = false,
            AppliedAt    = DateTime.UtcNow
        };
        _context.OperatorProfiles.Add(profile);
        _context.SaveChanges();
        break;

    case "Admin":
```

```
    case "Customer":
        // no extra setup required
        break;
}
```

8.3 Clean Folder Structure (Already Present, Reinforced Today)

The backend follows a layered architecture — each folder owns one layer of responsibility:

Models/ Data shape only — no logic, no DB calls

Controllers/ HTTP routing only — validates input, calls services/db, returns response

Services/ Business logic only — email, calculations, notifications

Interfaces/ Contracts only — no implementation, no dependencies

Data/ DB session only — DbContext and DbSets

8.4 What I understand better now

Coding standards are not about following rules for their own sake — they reduce cognitive load. When every method does one thing, I can read the code top to bottom and understand the entire booking workflow in minutes. The switch statement is key. That is real team communication through code.

10. Before vs After Applying OOP

Aspect	Before	After
Email service injection	Injected EmailService (concrete class)	Injects IEmailService (interface)
Email provider change	Edit BookingController + FeedbackController	Create new class, update Program.cs only
Role handling in Register	if (role == "Operator") {...}	switch(user.Role) — all roles in one block
Unit testing email	Hard — real SMTP called in tests	Easy — inject a mock IEmailService
Email method variants	One method — must always pass HTML body	Two overloads — quick notification possible
Interfaces folder	Did not exist	Backend/Interfaces/IEmailService.cs created
DI registration	AddScoped<EmailService>()	AddScoped<IEmailService, EmailService>()

Name – Sai Nikitha NSR
Day – 7 Assignment
Date – 4th May, 2026
Mail – sainikinsr@gmail.com

11. Summary — All OOP Concepts at a Glance

OOP Concept	File / Location	Status	Key Benefit
Encapsulation	Backend/Models/Booking.cs and all other model files	Already Present	Self-contained data objects; data integrity protected
Abstraction	Backend/Controllers/ Backend/Interfaces/IEmailService.cs	Present + Enhanced	Frontend/controllers hide DB and SMTP details completely
Inheritance	All 6 Controllers (: ControllerBase)	Already Present	HTTP handling reused from ASP.NET framework
Method Overriding	ControllerBase lifecycle hooks (OnActionExecuting)	Already Present	Custom cross-cutting logic per controller
Method Overloading	Backend/Interfaces/IEmailService.cs Backend/Services/EmailService.cs	NEW Added	Two SendBookingEmail signatures for different use cases
Interfaces	Backend/Interfaces/IEmailService.cs	NEW Added	Loose coupling; swap email provider without controller changes
Switch Statement	Backend/Controllers/UserController.cs	NEW Added	All role logic in one readable, extendable block
Service Layer	Backend/Services/EmailService.cs	Already Present	Email logic isolated; controllers stay clean
Clean Structure	Models/ Controllers/ Services/ Interfaces/ Data/ folders	Already Present	One responsibility per folder; easy navigation

12. Conclusion

The Bus Booking System already utilized some of the OOP concepts in the correct manner before today's discussion. Today's changes helped to complete the puzzle:

- Implementation of the IEmailService interface for loose coupling of controllers and email service
- Overloading was used for flexibility of the email API
- Replaced the chain of if by switch(user.Role) to make it more compact, readable, and extensible
- Created an Interfaces/ folder as per layered architecture design pattern

From today's experience, I have understood that OOP concepts are interconnected and should be implemented together. The encapsulation concept makes sure that models are trustable. Using abstraction concept in form of interfaces will make services easily swappable. Inheritance makes sure that controllers can handle HTTP requests. The polymorphism concept (overloading and overriding) will help in making the API flexible. Follow coding standards so that the whole application becomes very readable. This way we will have a codebase where each component will have one responsibility, everything will depend upon an interface, and anyone in future including ourselves will find it easy to understand the codebase.